

```
9  shared_var = shared_var - 1;
10 }
```

衍生场景一：读写场景并行读取

该场景为场景一的衍生场景。当多个线程同样需要访问同一共享数据，但多个线程只会读取共享数据时，允许这些线程并行（即同时）执行不会造成数据竞争，且能有效提升系统性能。

如代码片段 9.3 所示，若多个线程执行 reader 函数，其只读取共享的 shared_var 到局部变量 local_var。我们应该允许这些线程同时执行从而提升系统效率。而对于会修改数据的线程（执行 writer 函数），则需要保证没有其他线程在读写数据才能修改数据。

代码片段 9.3 衍生场景一：读写场景并行读取

```
1 int shared_var;
2 void reader(void)
3 {
4     local_var = shared_var;    // 只读取共享数据
5 }
6
7 void writer(void)
8 {
9     shared_var = shared_var + 1; // 修改共享数据
10 }
```

场景二：条件等待与唤醒

场景二与之前的场景不同，其不再关注如何协调线程对于共享资源的互斥访问，而是线程的执行顺序与条件（又称同步）。当线程需要等待某条件达成才能继续执行时，其应一直被阻塞直到条件达成。系统应提供对应的机制，在线程阻塞时调度到其他有计算需求的线程（即挂起该线程），避免浪费处理器以提升整体系统的效率。而在线程等待的条件达成后，系统应唤醒该阻塞等待（即挂起）的线程，让其能够继续执行。

如代码片段 9.4 所示，线程 2（执行 thread_2）现在需要等待线程 1（执行 thread_1）完成工作后再继续执行。操作系统应能够提供机制，在 thread_2 等待时调度到其他线程执行，最大限度地利用处理器的计算资源，并在线程 1 执行完毕之后（即条件达成）立刻唤醒线程 2 执行。

代码片段 9.4 场景二：条件等待与唤醒

```
1 int thread_1_state = UNFINISHED;
2 void thread_1(void)
3 {
4     doing_something();    // 完成当前线程的工作
5     thread_1_state = FINISH;
6     notify_thread_2();    // 通知线程 2 完成
```

```

7 }
8
9 void thread_2(void)
10 {
11     if (thread_1_state != FINISH)
12         wait();                // 等待线程 1 完成工作
13     doing_something();          // 完成线程 2 的工作
14 }

```

场景三：多资源协调管理

场景三是系统软件中十分常见的一种模式，其既包含对共享资源的管理，也包含协调多个不同线程的等待与执行。当多线程应用中存在多个资源可以被多个线程消耗或释放（生产）时，就需要协调这些线程有序获取资源或者等待。这里等待的条件就是有可用的资源，而唤醒的时机则是有线程释放（产生）了资源。

如代码片段 9.5 所示，当资源池 `shared_resources` 存在资源时，消费资源线程（执行 `consumer_thread` 函数）能成功获取资源并处理，否则将等待。生产资源线程（执行 `producer_thread` 函数）能向资源池生产（释放）新的资源，并且如果有线程在等待资源，可以通知这些线程获取资源并执行。

代码片段 9.5 场景三：多资源协调管理

```

1 struct resources *shared_resources;
2 void producer_thread(void)
3 {
4     produce(shared_resources); // 生产共享资源
5     notify_waiters();          // 通知等待的消费线程
6 }
7
8 void consumer_thread(void)
9 {
10     if (!has_resources(shared_resources))
11         wait();                // 等待资源
12     consume(shared_resources); // 消耗共享资源
13 }

```

9.2 同步原语

本节主要知识点

- ❑ 如何正确使用互斥锁、读写锁、条件变量与信号量？
- ❑ 针对不同的同步场景，如何选择合适的同步原语以满足同步需求？
- ❑ 不同同步原语有哪些不同及相似之处？
- ❑ 如何将多线程应用中的同步需求对应到同步场景，并使用合适的同步原语满足同步需求？

本节将结合具体的例子，识别多线程应用程序中存在的同步需求，归类到上一节介绍的典型场景，并使用四种不同的同步原语解决这些需求。表 9.1 展示了这些同步原语的功能描述与具体场景。最后，本节将对比这些同步原语，展示它们的异同。

表 9.1 四种同步原语的功能描述与具体场景

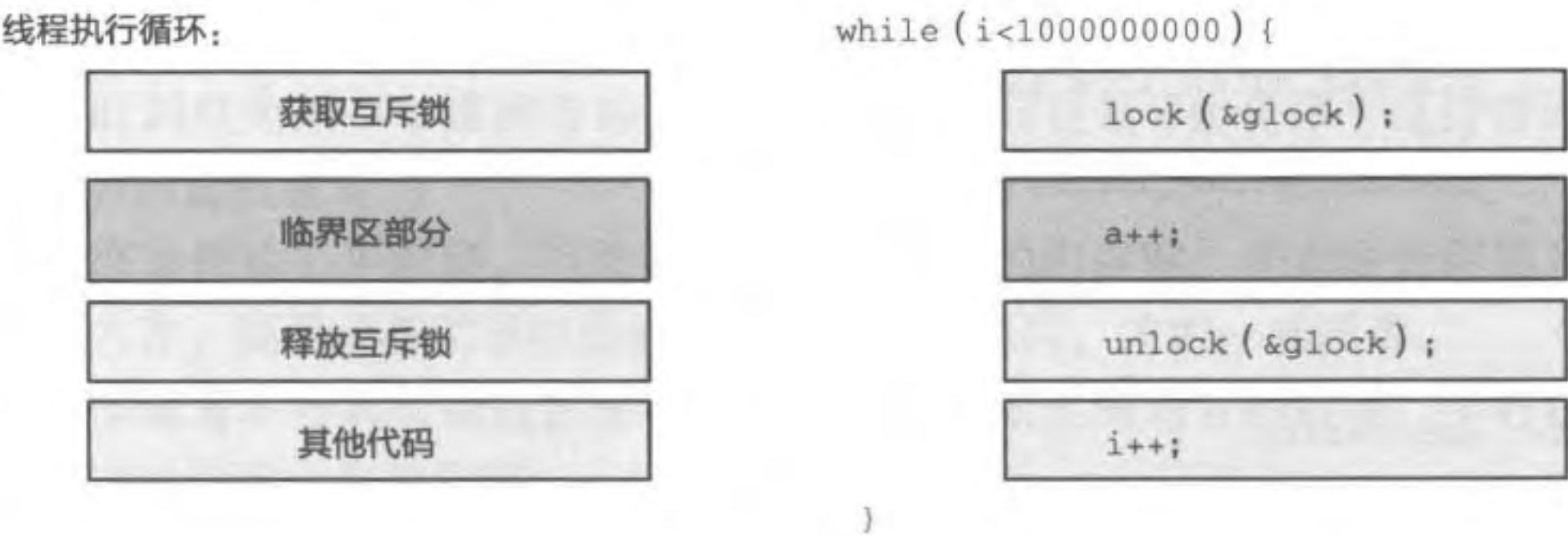
同步原语	描 述	场 景
互斥锁	保证对共享资源的互斥访问	场景一：共享资源互斥访问
读写锁	允许读者线程并行读取共享资源	衍生场景一：读写场景并行读取
条件变量	提供线程睡眠与唤醒机制	场景二：条件等待与唤醒
信号量	协调线程对有限数量资源的消耗与释放	场景三：多资源协调管理

9.2.1 互斥锁

在 9.1.1 节介绍的计数问题中，我们创建了两个线程，不断地在同一个全局共享计数器上进行加法操作。由于两个线程会同时修改计数器，发生了数据竞争，最终导致程序的执行结果与预期不符。这种程序的正确性依赖于特定执行顺序的情况也被称为竞争冒险 [Race Hazard，又被称为竞争条件 (Race Condition)]。比如在计数问题中，最后变量的数值不可预测且每次执行的结果都不一样。为了避免数据竞争并解决竞争冒险，我们需要防止多个线程同时访问同一地址，确保同一时刻只有一个线程能够访问该地址。

因此，该问题可以归类到上一节提出的“场景一：共享资源互斥访问”。针对这一类场景，互斥访问 (Mutual Exclusion) 可以保证同一时刻只有一个线程能够访问共享资源。而保证互斥访问共享资源的代码区域被称为临界区 (Critical Section)。在同一时刻，至多只有一个线程可以执行临界区中的代码。因此在临界区内，一个线程可以安全地对共享资源进行操作。如图 9.4 所示，在多线程计数问题中，对于全局的共享计数器 a 的修改就应该放在临界区中。如何设计特定机制来保证同一时刻只有一个线程可以执行临界区的问题被称为临界区问题。解决临界区问题是保证多线程应用程序正确性的关键。临界区问题不仅会在多核系统中出现，在单核系统中也同样存在：这是由于调度器允许多个线程在一个核心中交错执行，正在临界区执行的线程可能会被打断，然后调度到另一个线程再次进入临界区，从而造成两个线程同时处于临界区的现象，导致竞争冒险。

为了解决临界区问题，系统通常会提供互斥锁 (Mutual Exclusion Lock) 这种同步原语。互斥锁与生活中的换衣间门锁类似，其能够保证同一时刻只有唯一一位顾客进入。任何顾客如果需要进入换衣间，都需要确保门没有上锁，且在进入换衣间之后的第一时间上锁，确保不会有其他顾客进入同一换衣间。同理，我们可以为共享计数器设置一把锁，且只允许持有该锁的线程更新该计数器 (即执行临界区)，这样就可以保证多个线程不会同时更新计数器，从而消除数据竞争并解决临界区问题。



需要互斥访问的程序

例子：多线程计数

图 9.4 互斥访问与临界区：以多线程计数为例

代码片段 9.6 展示了互斥锁的接口。互斥锁提供两个易用的接口，包括 lock 与 unlock 操作。其中 lock 操作代表加锁，其将阻塞当前线程直到能够拿到该锁才能返回。而 unlock 操作代表放锁，其将释放该锁从而允许其他线程拿到锁。加锁操作与放锁操作之间的代码即临界区。临界区中的代码在同一时刻只允许一个线程执行（即互斥性）。例如，在图 9.4 中，我们可以通过在修改共享计数器的前后分别加入加锁与放锁操作，以保证对共享计数器 a 的互斥访问。

代码片段 9.6 互斥锁的接口

```
1 void lock(struct lock *mutex);
2 void unlock(struct lock *mutex);
```

代码片段 9.7 展示了修改后的多线程计数测试。这里使用的互斥锁是 pthread 库提供的互斥锁 pthread_mutex。为了保证同一时刻只有一个线程能够对共享的变量 a 进行操作，我们在访问其之前的第 9 行插入了一条加锁（即 pthread_mutex_lock）操作，而在修改完成后的第 11 行插入了一条放锁（即 pthread_mutex_unlock）操作。除了加锁与放锁之外，在使用互斥锁之前还需要使用 pthread_mutex_init（第 19 行）对该互斥锁进行初始化操作。读者可以在自己的机器上运行这份代码，其将稳定输出 2 000 000 000，而不会有其他的结果。通过简单的上锁与放锁，我们可以保证对共享计数器的互斥访问，从而避免由于数据竞争造成的错误结果。

代码片段 9.7 多线程计数测试：互斥锁版本

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 pthread_mutex_t global_lock;
5 unsigned long a = 0;
6 void *routine(void *arg)
7 {
```



```

8   for (int i = 0; i < 1000000000; i++) {
9       pthread_mutex_lock(&global_lock);
10      a++;
11      pthread_mutex_unlock(&global_lock);
12  }
13  return NULL;
14 }
15
16 int main(void)
17 {
18     pthread_t threads[2];
19     pthread_mutex_init(&global_lock, NULL);
20     for (int i = 0; i < 2; i++)
21         pthread_create(&threads[i], NULL, routine, 0);
22     for (int i = 0; i < 2; i++)
23         pthread_join(threads[i], NULL);
24     printf("%lu\n", a);
25     return 0;
26 }

```

除了加锁与放锁操作，互斥锁还提供了尝试加锁的 `try_lock` 操作。在调用加锁操作（即 `lock` 操作）时，其将阻塞当前线程直到成功上锁。而不同于加锁操作，`try_lock` 操作不会阻塞当前的线程。如果互斥锁此时为空闲，`try_lock` 操作将成功上锁并返回。否则，`try_lock` 不会阻塞并等待，而是直接返回，并通过返回值表示当前互斥锁不空闲，从而告知调用者加锁失败。`try_lock` 操作可以帮助线程更加灵活地使用互斥锁。无法成功上锁时，其允许线程执行其他逻辑，从而避免无用的阻塞。

练习

9.1 请改写代码片段 9.7，仅使用 `pthread_mutex_trylock` 而不使用 `pthread_mutex_lock` 实现相同的功能。`pthread_mutex_trylock` 返回 0 代表加锁成功，其他值代表加锁失败。

9.2.2 读写锁

现实中还存在另一种情况，也涉及对于共享资源的访问，但不再需要互斥特性。这里引入一个新的问题——公告栏问题来进一步阐释这种场景。如图 9.5 所示，假设现在有一个公告栏，一群人正在围观该公告栏的内容。现在根据能否修改公告栏内容将人群分成了两类不同的角色：维护者可以对公告栏内容进行维护，而围观者只能读取公告



图 9.5 公告栏问题

栏的内容而不能进行修改。该公告栏还有一系列准则需要遵守。

1. 同一时刻只允许一个维护者对公告栏的内容进行更新（避免多个维护者同时更新公告栏的内容而造成覆盖）。

2. 为了准确传达公告内容，当维护者更新公告栏的内容时，不允许任何围观者提前观看更新的内容。围观者必须等待维护者更新完公告栏后，才能一起围观。

3. 由于围观者不会对公告信息进行更改，因此不限定围观者的数量，所有在场的围观者都可以同时读取公告的内容。

为了实现以上准则，最简单的办法是利用互斥锁保证所有的维护者与围观者之间的互斥。这样既可以保证同一时刻只有一个维护者对公告栏进行更新（准则中的第一点），又可以保证围观者不会在维护者完成更新前提前读公告栏的内容（准则中的第二点）。然而，使用互斥锁会导致同一时刻只允许一个围观者读取公告栏（围观者之间也互斥），从而造成信息传递效率低下（没有充分利用准则三）。因此，我们需要一个新的同步原语，既能维护围观者与维护者之间的互斥性，也能够允许多个围观者同时读取公告栏内容。这个问题就是典型的“衍生场景一：读写场景并行读取”的例子。其中维护者就是写者，而围观者就是读者。下面我们将介绍一种新的同步原语以解决读者与写者问题。

读写锁（Reader Writer Lock）使用起来与互斥锁非常类似，不过读写锁针对读者与写者分别提供了不同的 lock 与 unlock 操作。如代码片段 9.8 所示，读者需要使用 lock_reader 与 unlock_reader 保护读临界区。而写者需要使用 lock_writer 与 unlock_writer 保护写临界区。当读者在执行读临界区的代码时，其可以保证没有其他写者在执行写临界区。而其他读者也可以同时执行读临界区的内容。写临界区既不允许其他读者进入读写锁保护的读临界区，也不允许其他写者进入读写锁保护的写临界区。因此，当一个读者需要上锁时，存在以下几种情况。

1. 没有其他读者 / 写者在读 / 写临界区内，此时读者可以直接进入读临界区。

2. 存在写者在写临界区内，此时读者不能进入读临界区，必须等待写者退出写临界区之后才能进入读临界区。

3. 存在读者在读临界区内，且没有其他写者需要进入写临界区，当前读者可以进入读临界区。

4. 存在读者在读临界区内，但存在其他写者需要进入写临界区。不同的读写锁在这里会采用不同的策略。偏向读者的读写锁允许在写者申请进入写临界区之后到达的读者优先进入读临界区，因此使用这种读写锁允许当前读者进入读临界区。而偏向写者的读写锁会阻塞在写者申请进入写临界区之后到达的读者，因此使用偏向写者的读写锁会阻塞当前的读者。我们将在第 10 章详细介绍这两种不同类型的读写锁的具体实现。

代码片段 9.8 读写锁使用示例

```
1 struct rwlock lock;
2 char data[SIZE];
3
4 void reader(void)
5 {
6     lock_reader(&lock);
7     read_data(data);    // 读临界区
8     unlock_reader(&lock);
9 }
10
11 void writer(void)
12 {
13     lock_writer(&lock);
14     update_data(data); // 写临界区
15     unlock_writer(&lock);
16 }
```

练习

9.2 下面的代码展示了三个不同的函数，它们均对全局共享的变量 `shared` 进行了操作。请判断哪些可以用读锁保护，哪些必须使用写锁保护。

```
1 int shared = 0;
2 int func_A(void)
3 {
4     return shared;
5 }
6
7 int func_B(void)
8 {
9     return shared++;
10 }
11
12 int func_C(void)
13 {
14     int ret = shared;
15     return ret++;
16 }
```

9.2.3 条件变量

在多线程应用程序中，除了需要保证互斥访问的场景外，还有另一类需要协调线程执行的需求。在本节中，我们将介绍一个计算机领域的经典问题：生产者 - 消费者问题。

生产者-消费者问题描述了一个多线程应用程序中常见的数据传递场景：假设我们有数个生产者线程与数个消费者线程，生产者不断地生产新的数据，而消费者不断地拿取并处理这些数据。它们之间通过一个有限容量的缓冲区共享数据。应用程序需要保证数据能够有序且正确地在生产者与消费者之间传递。如图 9.6 所示，在给出的例子中，生产者与消费者共享一个大小为 5 的缓冲区用于交换数据。生产者不断地将生成的新数据放到缓冲区中，与此同时，消费者从缓冲区拿取并消耗这些数据。图 9.6 中，缓冲区 0、1、2 分别存放着生产者放入缓冲区的数据，生产者 P_1 或 P_2 下一步产生的数据将放入 3 中，而消费者将从 0 中拿取数据进行处理。

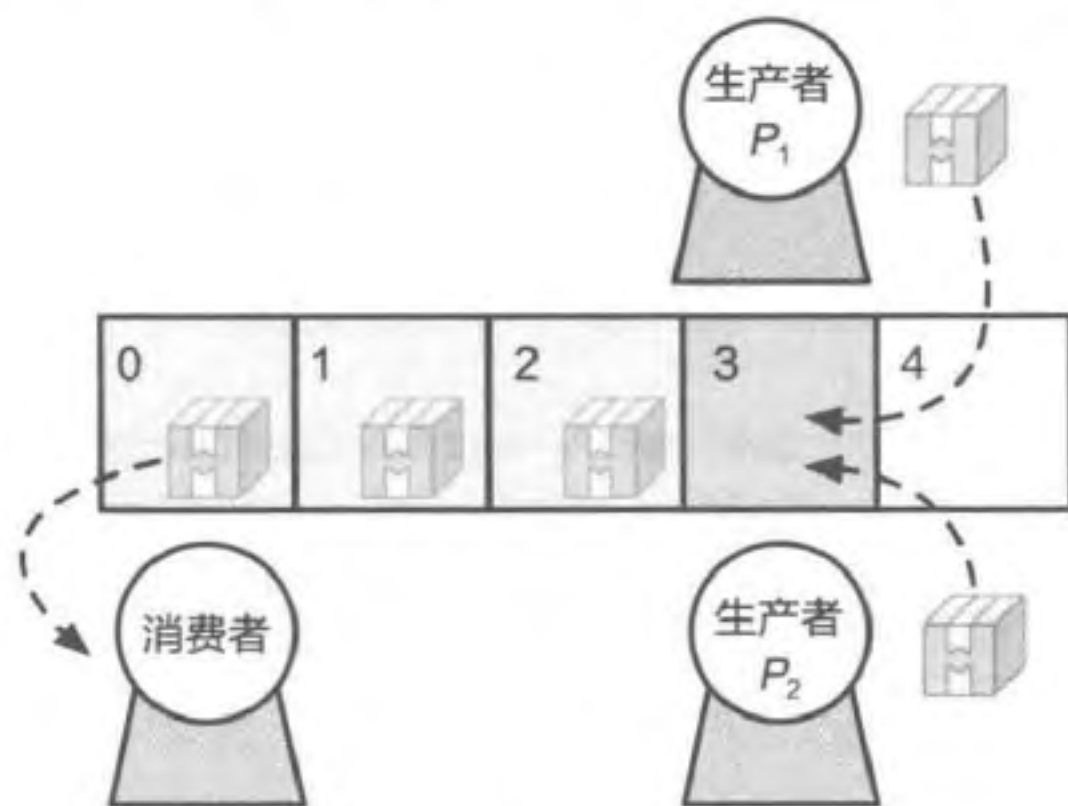


图 9.6 生产者-消费者问题

生产者-消费者模型在多线程应用程序中十分常见。如刚才列举的将海量数据划分到不同核心上的子任务进行处理的例子，我们最终需要收集这些子任务的处理结果。处理这些划分数据的线程是生产者，而收集处理结果的线程便是消费者。生产者-消费者问题中涉及两个不同的同步的需求：

- 数个生产者（或数个消费者）之间需要协同：需要避免因共享缓冲区中的数据产生数据竞争而造成错误。
- 生产者与消费者之间需要协同：消费者需要在缓冲区空时等待生产者生产数据，而生产者需要在缓冲区满时等待消费者消费数据。

针对第一个需求，即要在生产者之间（或消费者之间）正确协同，我们首先需要提供对于共享缓冲区的基础操作。代码片段 9.9 展示了对于共享缓冲区的两个操作。生产者将使用 `buffer_add` 操作向缓冲区加入新的数据，消费者将使用 `buffer_remove` 操作从缓冲区拿取数据。这两个操作均通过各自的计数器来选择需要放入以及读取的位置。例如，当处于图 9.6 的情景时，全局的 `buffer_write_cnt` 为 3。当生产者调用 `buffer_add` 操作向缓冲区加入新数据时，会选择位置 3 放入新的数据并更新 `buffer_write_cnt`。若此时没有其他生产者同时调用 `buffer_add` 操作，该算法可以正确记录当前的生产进度。然而若生产者 P_1 与 P_2 同时调用 `buffer_add` 操作，如果不加干预， P_1 与 P_2 可能同时执行第 7 行代码，选择 3 号缓冲区来放入自己的数据，最终导致数据覆盖，出现错误。因此，第一个需求可以归类到场景一。根据 9.2.1 节的介绍，我们可以使用互斥锁来保证对共享缓冲区的访问。

代码片段 9.9 有限缓冲区操作的初步实现

```
1 volatile int buffer_write_cnt = 0;
2 volatile int buffer_read_cnt = 0;
```



```
3 int buffer[5];
4
5 void buffer_add(int msg)
6 {
7     buffer[buffer_write_cnt] = msg;
8     buffer_write_cnt = (buffer_write_cnt + 1) % 5;
9 }
10
11 int buffer_remove(void)
12 {
13     int ret = buffer[buffer_read_cnt];
14     buffer_read_cnt = (buffer_read_cnt + 1) % 5;
15     return ret;
16 }
```

练习

9.3 如果只有一个生产者但是有多个消费者，继续使用代码片段 9.9 对共享缓冲区进行操作，是否会出现正确性问题？请尝试使用例子进行说明。

代码片段 9.10 展示了如何使用互斥锁保护生产者和消费者的共享缓冲区。与代码片段 9.9 相比，这里添加了一把全局的互斥锁 `buffer_lock`。在初始化缓冲区时，使用 `lock_init` 初始化该互斥锁。这里为了保证对于共享缓冲区的互斥访问，在 `buffer_add` 与 `buffer_remove` 操作开始前使用 `lock` 拿到全局互斥锁，结束后使用 `unlock` 释放该互斥锁。这样，所有对于共享缓冲区的操作均被放置在加锁与放锁之间的临界区内，保证了程序的正确性。

代码片段 9.10 利用互斥锁保护生产者和消费者缓冲区的操作

```
1 volatile int buffer_write_cnt = 0;
2 volatile int buffer_read_cnt = 0;
3 struct lock buffer_lock;
4 int buffer[5];
5
6 void buffer_init(void)
7 {
8     lock_init(&buffer_lock);
9 }
10
11 void buffer_add_safe(int msg)
12 {
13     lock(&buffer_lock);
14     buffer[buffer_write_cnt] = msg;
15     buffer_write_cnt = (buffer_write_cnt + 1) % 5;
16     unlock(&buffer_lock);
```

```
17 }
18
19 int buffer_remove_safe(void)
20 {
21     lock(&buffer_lock);
22     int ret = buffer[buffer_read_cnt];
23     buffer_read_cnt = (buffer_read_cnt + 1) % 5;
24     unlock(&buffer_lock);
25     return ret;
26 }
```

针对第二个需求，为了维护生产者和消费者之间的正确性，需要满足以下两个前提条件：当缓冲区已满时，生产者应停止向缓冲区写入数据，避免数据被覆盖并等待消费者消费缓冲区内的数据；当缓冲区为空时，消费者应停止从缓冲区拿取数据，避免读取到无效数据并等待生产者生成新的数据。

我们首先尝试使用两个计数器来解决这个问题。如代码片段 9.11 所示，设置了两个全局的计数器 `filled_slot` 与 `empty_slot`，并使用两个互斥锁 `filled_cnt_lock` 与 `empty_cnt_lock` 保护这两个计数器。其中 `filled_slot` 用于记录在缓冲区中可以被消费的对象数量，而 `empty_slot` 用于记录在缓冲区中剩余的空位数量。如图 9.6 所示，`filled_slot` 与 `empty_slot` 分别为 3 与 2，代表在缓冲区中还有 3 个未消耗的对象（缓冲区 0、1、2），而缓冲区还有 2 个空位。当生产者需要将新产生的数据放入缓冲区时，需要检查缓冲区中剩余的空位数量 `empty_slot`。当缓冲区没有空位时，生产者需要等待直到出现空位（第 12 行）。而在生产者成功将数据放入缓冲区后，需在减少空位数量 `empty_slot`（第 16 行）的同时，增加可消费的对象数量 `filled_slot`（第 20 行）。同样，当消费者需要从缓冲区拿取对象时，需要检查可消费的对象数量 `filled_slot`。如果该计数器为 0，就代表没有可以被消耗的对象，此时消费者需要等待（第 30 行）。而在消费者成功拿到对象后，需在减少 `filled_slot`（第 34 行）的同时，增加用于标记空位数量的 `empty_slot`（第 38 行）。通过这样一个简单的基于计数器的算法，我们可以正确地协调生产者与消费者对缓冲区的操作，避免产生错误。

小思考

为何需要在判断是否有空位 / 是否有资源之前就获取互斥锁（例如第 12 行之前），而不是仅仅在修改对应计数器时再获取互斥锁？

其主要原因是判断是否有空位以及消耗空位的两个操作应当在一个临界区内，确保同一时刻只能有一个线程发现有空位并消耗这个空位。若将判断放在临界区外，则有可能出现两个线程同时发现 `empty_slot_cnt` 不为 0，然后均退出循环并尝试获

取锁，接着进入第 16 行并修改计数器，最终造成错误。此外，在等待空位时，线程将不断获取与释放互斥锁。这是由于消费者产生空位时也需要获取对应的互斥锁。如果在等待时不释放互斥锁，消费者也无法进入临界区，从而无法更新计数器，造成生产者无限等待下去。

可以发现，在协调生产者和消费者时，我们需要让生产者在满足“缓冲区有空位”这个条件之前一直等待，直到有消费者消费资源并产生了空位，才允许生产者继续执行。对于消费者也是如此。这就出现了之前总结的经典场景中的“场景二：条件等待与唤醒”。虽然代码片段 9.11 能够正确协调生产者与消费者，但无论是生产者还是消费者，在没有空位 / 资源时就会陷入循环等待（Busy Looping，也称为循环忙等），如第 12 ~ 14 行所示。而在消费者释放空位之前，循环等待是毫无意义的。这些无谓的循环不仅浪费了 CPU 资源，还增加了系统的能耗。因此，我们希望使用一种挂起唤醒的机制来解决这个问题。

代码片段 9.11 利用计数器协调生产者与消费者

```
1 volatile int empty_slot = 5;
2 volatile int filled_slot = 0;
3 struct lock empty_cnt_lock;
4 struct lock filled_cnt_lock;
5
6 void producer(void)
7 {
8     int new_msg;
9     while(TRUE) {
10         new_msg = produce_new();
11         lock(&empty_cnt_lock);
12         while (empty_slot == 0) {
13             unlock(&empty_cnt_lock);
14             lock(&empty_cnt_lock);
15         }
16         empty_slot--;
17         unlock(&empty_cnt_lock);
18         buffer_add_safe(new_msg);
19         lock(&filled_cnt_lock);
20         filled_slot++;
21         unlock(&filled_cnt_lock);
22     }
23 }
24
25 void consumer(void)
26 {
27     int cur_msg;
28     while(TRUE) {
```

```
29     lock(&filled_cnt_lock);
30     while (filled_slot == 0){
31         unlock(&filled_cnt_lock);
32         lock(&filled_cnt_lock);
33     }
34     filled_slot--;
35     unlock(&filled_cnt_lock);
36     cur_msg = buffer_remove_safe();
37     lock(&empty_cnt_lock);
38     empty_slot++;
39     unlock(&filled_cnt_lock);
40     consume_msg(cur_msg);
41 }
42 }
```

条件变量 (Condition Variable) 提供了一系列原语, 使得当前线程在等待的某一条件不能满足时停止使用 CPU 并将自己挂起。在挂起状态, 操作系统调度器不再选择当前线程执行。直到等待的条件满足, 其他线程才会唤醒该挂起的线程继续执行。使用条件变量能够有效避免无谓的循环等待。比如在生产者和消费者的例子中, 使用条件变量可以让生产者等待在“是否还有空位”这一条件上, 待到消费者发现已经有新的空位之后再将其唤醒, 从而避免生产者无意义的循环等待。下面我们将以生产者和消费者为例介绍如何使用条件变量。

如代码片段 9.12 所示, 条件变量提供了三个接口: `cond_wait` 用于挂起当前线程以等待在对应的条件变量上, `cond_signal` 用于唤醒一个等待在该条件变量上的线程, `cond_broadcast` 则用于唤醒所有等待在该条件变量上的线程。条件变量必须搭配一个互斥锁一起使用。该互斥锁用于保护对条件的判断与修改。`cond_wait` 接收两个参数, 分别为条件变量与搭配使用的互斥锁。在调用 `cond_wait` 时, 需要保证当前线程已经获取搭配的互斥锁 (在临界区中)。`cond_wait` 在挂起当前线程的同时, 该互斥锁同时被释放。其他线程则有机会进入临界区, 满足该线程等待的条件, 然后利用 `cond_signal` 唤醒该线程。挂起线程被唤醒之后, 会在 `cond_wait` 返回之前重新获取互斥锁。`cond_signal` 只接收一个参数: 当前使用的条件变量。虽然 `cond_signal` 没有明确要求必须持有搭配的互斥锁 (在临界区中), 但在临界区外使用 `cond_signal` 需要非常小心才能保证不丢失唤醒。我们将在下面的例子中进行详细解释。`cond_broadcast` 与 `cond_signal` 类似, 唯一的区别是 `cond_broadcast` 会唤醒所有等待在该条件变量上的线程, 而 `cond_signal` 只会唤醒其中的一个。

代码片段 9.12 条件变量的接口

```
1 void cond_wait(struct cond *cond, struct lock *mutex);
2 void cond_signal(struct cond *cond);
3 void cond_broadcast(struct cond *cond);
```


小思考

为何 `cond_wait` 需要传入其搭配的互斥锁 `mutex`？不能释放了之后再等待吗？

其本质原因是必须保证放锁操作和挂起操作一起发生。试想，如果我们在 `cond_wait` 之前就释放了互斥锁，则可能出现以下情况。假设存在线程 A 与线程 B。线程 A 在等待线程 B 更新某条件，线程 A 在挂起之前释放了互斥锁。而此时线程 B 拿到了互斥锁并更新了条件，调用 `cond_signal` 唤醒等待的线程。由于线程 A 还未挂起，因此其错过了该次唤醒。等到线程 A 挂起时，可能系统中不再会有线程来唤醒线程 A，导致出现错误。因此，`cond_wait` 通过操作系统保证了释放和挂起两个操作同时发生，从而避免了这种情况的发生。

代码片段 9.13 展示了如何在生产者与消费者中使用条件变量避免循环等待，与本节开头实现的生产者（代码片段 9.11）相比，使用条件变量的生产者主要有以下三点区别：

- 针对生产者与消费者等待的条件，这里创建了两个不同的条件变量 `empty_cond` 与 `filled_cond`，分别对应“缓冲区无空位”和“缓冲区无数据”两个条件。
- 当生产者发现没有空闲位置时，就会使用 `cond_wait` 进入休眠状态，避免循环等待（第 15 行）。由于生产者需要等待到 `empty_slot` 不为 0 时（即缓冲区有空位）才能向缓冲区填入新的内容，因此这里等待的条件变量为 `empty_cond`。当有新的空位产生时，消费者会利用 `cond_signal` 操作唤醒等待在 `empty_cond` 上的生产者（第 42 行）。
- 相应地，当消费者发现已经没有待消耗的内容时（第 33 行），也会使用 `cond_wait` 进入休眠状态（第 34 行）。这里等待的是表示是否有待消耗内容的条件变量 `filled_cond`。当生产者生产新的内容供消费者消耗时，其同样会使用 `cond_signal` 操作唤醒等待在 `filled_cond` 上的消费者（第 23 行）。

代码片段 9.13 生产者 - 消费者问题的条件变量

```

1 volatile int empty_slot = 5;
2 volatile int filled_slot = 0;
3 struct cond empty_cond;
4 struct lock empty_cnt_lock;
5 struct cond filled_cond;
6 struct lock filled_cnt_lock;
7
8 void producer(void)
9 {
10     int new_msg;
11     while(TRUE) {
12         new_msg = produce_new();
13         lock(&empty_cnt_lock);
14         while (empty_slot == 0)

```

```
15     cond_wait(&empty_cond, &empty_cnt_lock);
16     empty_slot--;
17     unlock(&empty_cnt_lock);
18
19     buffer_add_safe(new_msg);
20
21     lock(&filled_cnt_lock);
22     filled_slot++;
23     cond_signal(&filled_cond);
24     unlock(&filled_cnt_lock);
25 }
26 }
27
28 void consumer(void)
29 {
30     int cur_msg;
31     while(TRUE) {
32         lock(&filled_cnt_lock);
33         while (filled_slot == 0)
34             cond_wait(&filled_cond, &filled_cnt_lock);
35         filled_slot--;
36         unlock(&filled_cnt_lock);
37
38         cur_msg = buffer_remove_safe();
39
40         lock(&empty_cnt_lock);
41         empty_slot++;
42         cond_signal(&empty_cond);
43         unlock(&empty_cnt_lock);
44         consume_msg(cur_msg);
45     }
46 }
```

读者可能会对第 14 行使用 while 循环抱有疑问：当一个线程被唤醒时，其等待的条件理应被满足了，为何还需要再次检查？这里我们通过一个例子解释再次检查的必要性。如图 9.7 所示，现在存在一个消费者（线程 2）与两个生产者（线程 1 与线程 3）。在某一时刻，empty_slot 的值为 0，代表目前没有空位。因此当线程 1 的生产者执行到第 14 行时，会由于没有空位而使用 cond_wait 进入睡眠。之后，线程 2（可以运行在同一核心或不同核心）的消费者消耗了一个对象，并产生了一个空位。此时 empty_slot 的值变为 1（执行第 41 行），并使用 cond_signal 来唤醒线程 1。然而，在线程 1 唤醒之前，另外一个生产者线程 3（同样可以执行在同一核心或者不同核心）开始执行生产者流程。由于此时的 empty_slot 的值为 1，因此该线程不会进入睡眠，而是直接消耗掉该空位（第 16 行）。待到线程 1 重新被调度并成功获取互斥锁之后，此时 empty_slot 的值已经被线程 3 减为 0 了。如果不使用循环再次检查，线程 1 在这种“狼来了”的情况下就会错误地将 empty_slot 修改为负值（直接执行第 16 行代码）。

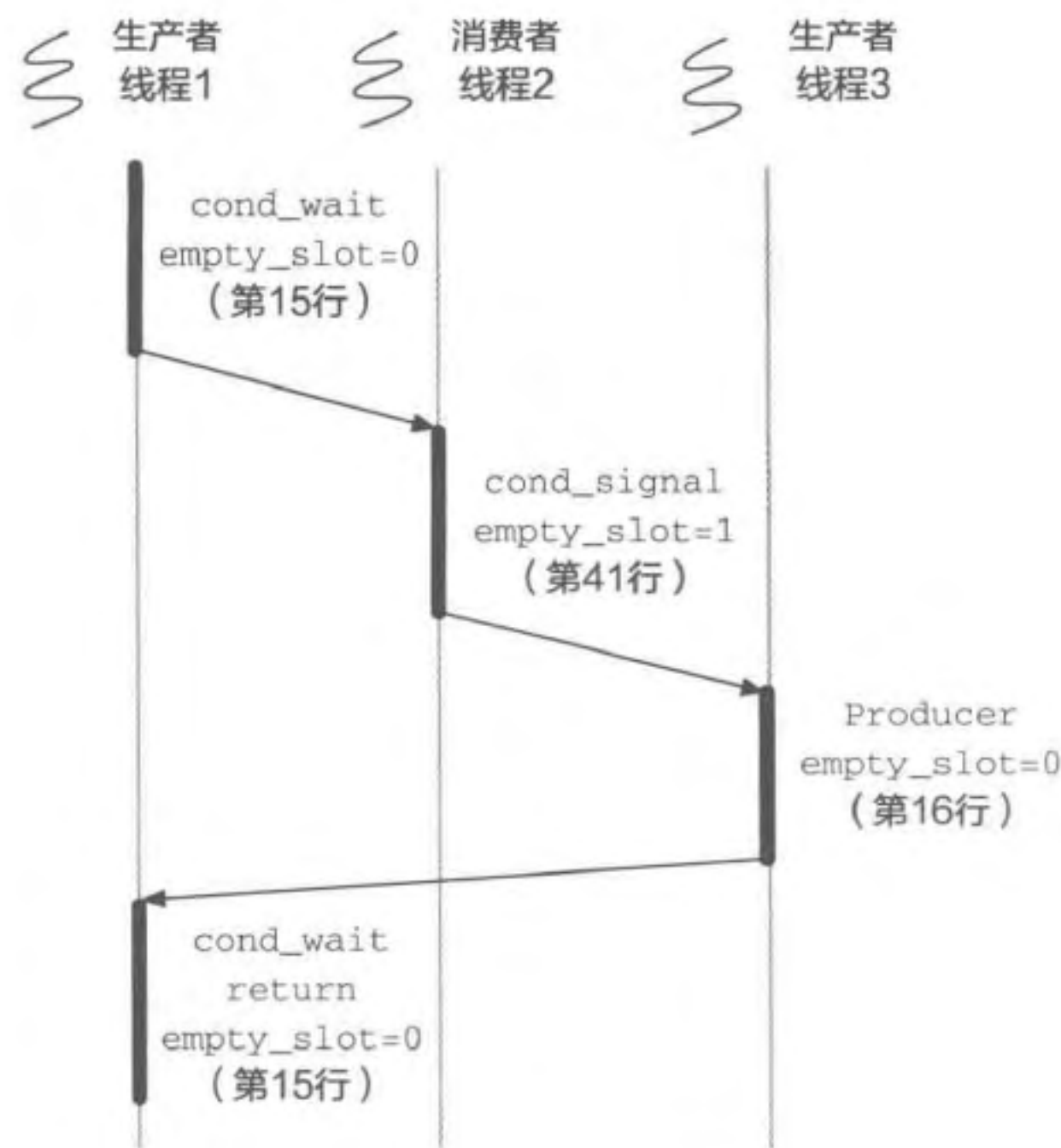


图 9.7 一种特殊情况

小思考

这里生产者每次都调用 `cond_signal` 是否有必要？

一种错误的想法是只有在 `filled_slot` 为 0 时才会有消费者等待在该条件变量上，因此只有在这种情况下才需要调用 `cond_signal`。然而，在 `filled_slot` 为 0 时，可能会有多个消费者等待。如果此时有多个生产者到达，只在 `filled_slot` 为 0 时调用一次 `cond_signal` 只能唤醒等待的消费者中的一个，其他的消费者将陷入无限的等待。一个可选的方案是使用 `cond_broadcast` 在 `filled_slot` 为 0 时唤醒所有的等待者。与 `cond_signal` 相比，主要区别在于 `cond_broadcast` 会唤醒所有等待的消费者。而最终只有一个消费者能够成功获取新产生的对象，其他消费者都将在重新检查过后再次进入睡眠，从而浪费一定的 CPU 资源。这种情况在消费者较多而生产者较少时更为严重。我们称这种现象为惊群效应：每当一个新的内容来临时，都会唤醒一群等待者，而最终只有其中的一个等待者可以成功进入临界区。

小思考

`cond_signal` 能否放在临界区外？

在代码片段 9.13 中，`cond_signal` 均被放置在临界区内。如果将其向下移，放

到临界区外，不会对正确性造成影响。以生产者为例，如果我们将生产者中第 23 行的 `cond_signal` 挪至第 24 行之后，当生产者执行到 `cond_signal` 时，其已经完成对 `filled_slot` 的更新操作并放锁。在该线程拿到互斥锁之前（第 21 行）调用 `cond_wait` 的消费者，可以正常被 `cond_signal` 唤醒，而在该线程放锁之后（第 24 行）进入临界区的消费者会发现 `filled_slot` 已经被更新，从而不会调用 `cond_wait`。然而在有些情况中，临界区外的 `cond_signal` 可能会面临丢失唤醒的问题。比如将代码片段 9.13 中的第 21 行与第 24 行去除（即代码片段 9.14 中的第 4 行与第 9 行），利用原子操作对 `filled_slot` 进行更新。这里原子操作是指对 `filled_slot` 的更新通过硬件机制一次性可见，从硬件层面避免数据竞争的出现，从而无须使用额外的互斥锁保护 `filled_slot`。原子操作将在第 10 章进行详细介绍，这里可以简单认为是一个对 `filled_slot` 的自增操作。那么可能出现按照以下顺序依次执行而导致消费者陷入无限等待的情况：

1. 消费者执行第 33 行代码，发现 `filled_slot` 为 0。
2. 生产者执行对 `filled_slot` 的自增操作，并调用 `cond_signal`。
3. 消费者调用第 34 行的 `cond_wait` 操作。

由于生产者调用 `cond_signal` 时并没有线程等待在该条件变量上，因此不能唤醒任何线程。而在消费者真正调用 `cond_wait` 挂起等待时，已经没有生产者再来唤醒该线程了。这时，生产者就产生了丢失唤醒的问题。因此，`cond_signal` 一般在临界区内使用，而在临界区外调用 `cond_signal` 时需要格外小心。

代码片段 9.14 修改后的生产者代码

```

1 void producer(void)
2 {
3     // ...
4     // - lock(&filled_cnt_lock);
5     // - filled_slot++;
6     // 原子地对 filled_slot 加 1
7     atomic_FAA(&filled_slot, 1); // 添加该行
8     cond_signal(&filled_cond);
9     // - unlock(&filled_cnt_lock);
10 }
```

9.2.4 信号量

上一小节解决了多生产者多消费者在有限缓冲区上的协同问题，满足了协调多个生产者（消费者）之间（场景一）以及协调生产者与消费者之间（场景二）的同步需求。而实际上，如果我们将缓冲区的空位当作一种资源，消费者需要消耗空位这个资源，而生产者可以产生空位资源，那么生产者与消费者问题同样可以归到“场景三：多资源协调

管理”中。针对这一类衍生场景，有一类更高层级抽象的同步原语能更加方便地解决同步需求。在之前的实现中，用于记录有多少个空位与多少个资源的计数器本质上充当了阻塞与继续执行的“风向标”，这也正是本节将介绍的同步原语信号量（Semaphore）的核心功能。信号量在不同的线程之间充当信号灯的作用，其根据剩余资源数量控制不同线程的执行或等待。需要注意的是，这并不意味着信号量比条件变量更胜一筹，而是其在该特定衍生场景中，能够更加便捷地满足同步需求。对于其他需要等待/唤醒操作的场景（比如更加复杂的场景），开发者还是需要使用条件变量来同步。我们将在本节的末尾比较这些同步原语，届时会更加清晰地对比这些同步原语的异同。

信号量最早由荷兰计算机科学家、图灵奖得主 Edsger Dijkstra 在 20 世纪 60 年代提出^[7]。除了初始化之外，信号量只能通过两个操作来进行更新：P 操作，源自荷兰语 *Proberen*，即“检验”；V 操作，源自荷兰语 *Verhogen*，即“自增”。正因为如此，信号量又被称为 PV 原语。为了便于理解，一般会使用 `wait` 和 `signal` 来表示信号量的 P 操作和 V 操作。

代码片段 9.15 给出了 `wait` 和 `signal` 操作的具体语义。其中 `wait` 操作用于等待。当信号量的值（代表资源数量）小于等于 0 时进入循环等待（第 3 行）。只有在信号量的值大于 0 时，才停止等待并消耗该资源（第 5 行，减少信号量的值）。`signal` 操作用于通知，其会增加信号量的值供 `wait` 的线程使用。需要注意的是，代码片段 9.15 只给出了信号量的语义，如果需要保证多个线程并行执行的正确性，还需要对代码进行很多修改。我们将第 10 章介绍信号量的具体实现，这里先关注信号量的使用场景和使用方法。

代码片段 9.15 信号量语义

```
1 void wait(int *S)
2 {
3     while(*S <= 0)
4         ; // 循环忙等
5     *S = *S - 1;
6 }
7
8 void signal(int *S)
9 {
10    *S = *S + 1;
11 }
```

代码片段 9.16 展示了如何使用信号量的标准接口解决生产者 - 消费者问题。在生产者 - 消费者模型中，两个信号量分别代表两个共享的资源：`empty_slot` 代表空余缓冲区的资源，而 `filled_slot` 代表已填入内容的缓冲区资源。生产者在该问题中会消耗空余缓冲区的资源并增加已填入的缓冲区资源，因此其在处理流程中应当先等待空余缓冲区的资源（第 9 行），而在填入内容后增加已填入的缓冲区资源数量（第 11 行）。消费者则刚好相反。

代码片段 9.16 利用信号量解决生产者 - 消费者问题

```

1 sem_t empty_slot;
2 sem_t filled_slot;
3
4 void producer(void)
5 {
6     int new_msg;
7     while(TRUE) {
8         new_msg = produce_new();
9         wait(&empty_slot); // P
10        buffer_add_safe(new_msg);
11        signal(&filled_slot); // V
12    }
13 }
14
15 void consumer(void)
16 {
17     int cur_msg;
18     while(TRUE) {
19         wait(&filled_slot); // P
20         cur_msg = buffer_remove_safe();
21         signal(&empty_slot); // V
22         consume_msg(cur_msg);
23     }
24 }

```

除了 wait 与 signal 操作以外，另一个可能会修改信号量的操作是赋予信号量初值。结合生产者 - 消费者的例子，当生产者与消费者均未开始工作且缓冲区中没有填入数据之时，空余的缓冲区的数量等于缓冲区的大小，而已填入内容的缓冲区数量为 0。因此针对 empty_slot 与 filled_slot 的初值，我们应该分别填入缓冲区大小与 0。

结合生产者 - 消费者的例子，我们可以总结出：信号量是用来辅助控制多个线程访问有限数量的共享资源。信号量只允许三个不同的操作对其值进行修改，分别是初始化操作、wait 操作与 signal 操作。信号量的初值应当设置为初始共享资源的数量。尝试消耗共享资源的线程应当调用 wait 操作来等待资源就绪，而产生或释放共享资源的线程应当调用 signal 操作来通知资源就绪。

练习

9.4 下面是两个内核中的功能，现在假设内核提供内核信号量，以及 sem_init、sem_wait 与 sem_signal 三个接口。其中 sem_init 将创建返回内核的信号量（初始值赋为 0），其可以传输给其他进程使用。下列功能如何实现？请使用伪代码描述。

(a) 在关于进程的章节中，我们介绍了 waitpid，它将阻塞当前进程，一直等

到目标进程结束退出。请描述如何实现。

- (b) 除了 `waitpid` 以外，系统还提供了 `sleep`，它将阻塞当前进程直到指定的时间结束。请思考如何配合**定时器** (timer) 使用内核信号量实现该功能。

9.2.5 同步原语的比较

本小节将详细对比前文介绍的四种同步原语。需要注意的是，这些同步原语并非完全正交，也不存在谁比谁更优的情况。总的来说，这些同步原语都用于正确地协同多个不同线程之间的执行。针对不同的场景，选用合适的同步原语能够更加方便地达成目标并避免错误。场景一与衍生场景一使用的互斥锁与读写锁均是为了保证对共享数据的修改（即互斥）；场景二使用的条件变量则是用于协调线程执行的顺序与条件（即同步）；而场景三使用的信号量则兼顾了对共享资源的管理与线程执行顺序的协调，既可以用于保证互斥，又可以用于在线程之间同步。我们将在下面结合具体例子进行展示。

我们先来对比互斥锁与读写锁。读写锁用于在读写场景中支持并行读取（衍生场景一）。正如在之前介绍读写锁时所述，直接使用互斥锁同样可以保证该场景的正确性，但是由于大量读者无法同时读取，会导致性能问题。因此，读写锁只适用于存在读者的场景，其在有大量读者需要执行读临界区时，能够提供比互斥锁更好的性能。后续章节不再将读写锁与其他同步原语进行比较。

互斥锁与条件变量的区别更加明显。互斥锁是用于解决临界区问题的，保证互斥访问共享资源（场景一）。而条件变量是通过提供挂起 / 唤醒机制来避免循环等待的，可节省 CPU 资源（场景二）。条件变量需要和互斥锁搭配使用。此外，场景一中也存在场景二的需求。如果我们将场景一中是否有线程在执行临界区作为等待条件，则可以使用条件变量让同一时刻无法进入临界区的线程挂起，从而避免循环等待。我们将在下一章具体讨论这种实现。

互斥锁与信号量有相似之处。当信号量的初值设置为 1（即将临界区看作唯一的资源），且只允许其值在 0 和 1 之间变化时，`wait` 与 `signal` 操作分别与互斥锁的 `lock` 与 `unlock` 操作类似。我们称这种信号量为二元信号量。二元信号量与互斥锁的差别在于：互斥锁有拥有者这一概念^①，而二元信号量没有。互斥锁往往是由同一个线程加锁和放锁，而信号量允许不同线程执行 `wait` 与 `signal` 操作。互斥锁与计数信号量（非二元信号量的其他信号量）区别较大。计数信号量允许多个线程通过，其数量等同于剩余可用资源数量；而互斥锁同一时刻只允许一个线程获取。互斥锁用于保证多个线程对于

^① 注意互斥锁仅仅是语义上有这个要求，实际实现中有的互斥锁可能不会对拥有者进行进一步检查。代码片段 10.11 实现读写锁时，`writer_lock` 可能被不同的线程获取与释放，因此在选择该互斥锁的实现时，需要注意互斥锁是否有针对拥有者的检查，或者直接使用二元信号量代替。

一个共享资源（即临界区）的互斥访问，而信号量则是用于协调多个线程对一系列共享资源的有序操作。例如，互斥锁可以用于保护生产者消费者问题中的共享缓冲区，而信号量用于协调生产者与消费者何时该等待，何时该放入或拿取缓冲区数据。

条件变量与信号量提供了类似的操作接口（包括 `wait` 与 `signal`），因此很容易混淆。实际上在很多情况下（比如本节的生产者消费者问题），条件变量与信号量都能够满足同步需求。它们之间的区别是层级之间的区别。条件变量提供了一个操作系统辅助的睡眠与唤醒机制（场景二），而信号量则是针对其中一个特定的场景（场景三），提供了对有限资源管理更加方便易用的接口供开发者使用。在生产者和消费者的实现中，开发者同样可以使用“互斥锁 + 计数器 + 条件变量”实现与信号量同样的功能。但对于某些其他场景，信号量就不再方便使用，也即条件变量的适用范围更加广泛。一个比较直观的例子是，针对事务处理场景，如果处理线程等待事务到达后希望一次性拿走所有的待处理事务一并处理，使用信号量就很难实现——信号量每次执行 `wait` 操作只能拿走其中的一个资源。总而言之，条件变量能够在更广泛的场景中使用，而信号量则是管理有限资源时更方便的抽象。

练习

9.5 下面是一系列系统软件中可能会遇到的场景，请将其归纳到本章介绍的场景中，并阐释如何使用本章介绍的同步原语解决问题。

- (a) 多线程应用程序中常用到的一种工具叫作多线程执行屏障。其作用如下：假设现在有多个线程同时运行，运行到执行屏障后，线程不再继续往下运行，而是等待所有的线程都执行到该屏障之后再继续执行。如何协同这些多线程实现执行屏障的功能？
- (b) 操作系统多核调度器的每个核心都有自己的等待队列。CPU 核心将在自己的等待队列中选取合适的线程执行。现在需要一种支持工作窃取的负载均衡机制：当自己的等待队列为空时，可以去其他核心等待队列窃取任务执行，从而达到负载均衡。如何协调多个核心上的调度线程？
- (c) 并行计算程序通常使用一种映射 - 归约（Map Reduce）模式将较大的任务拆分成多个可以并行执行的小任务，然后将这些小任务的计算结果合并起来成为最终结果。比如在并行程序统计字符数的场景下，会创建多个工作线程用于拆分统计某大文件中的字符数。工作线程执行完成后由一个收集线程收集每个工作线程统计的字数并累加。工作线程完成时间不等，一旦有任何一个工作线程完成执行，收集线程就应该开始累加，而不是等待全部执行完成后再累加。如何协调工作线程与收集线程？
- (d) 某网页渲染时需要同时发多个请求，并等待全部请求都成功后再完成页面渲

染。这些请求都带有回调函数。如何在回调函数中采用同步原语完成该需求？

(e) 某服务器需要控制处理请求线程的并行数量，避免过多线程同时运行。如何协调这些线程的执行？

(f) 网页服务器中，有的线程用于响应客户端获取网页的需求，有的线程用于后端实时更新网页。如何协调这些线程对于网页数据的访问？

9.3 死锁

本节主要知识点

- 发生死锁的四个必要条件是什么？
- 如何检测一个系统中是否存在死锁？
- 在发生死锁后如何恢复到正常状态？
- 如何在源头上预防死锁？
- 如何使用银行家算法在运行时避免产生死锁？

9.3.1 死锁的定义

同步原语在给开发者带来便利的同时，也带来了一系列问题。本节将介绍其中最为经典的问题：死锁问题。下面我们通过一个简单的例子来介绍何为死锁。假设线程 A 与线程 B 分别运行代码片段 9.17 中的 `thread_A` 与 `thread_B`。这两个线程共享两把全局的互斥锁 `lock_A` 与 `lock_B`。线程 A 先尝试获取锁 A（第 5 行），再尝试获取锁 B（第 7 行），而线程 B 刚好相反。假设在 T_0 时刻，线程 A 获取了锁 A（第 5 行），将要尝试获取锁 B。而线程 B 刚好也获取了锁 B（第 15 行），等待获取锁 A。但是由于两个线程都持有对方将要获取的锁，因此这两个线程都不能进入临界区继续执行，这种互相等待的僵持状态即死锁现象。当有多个（两个及以上）线程竞争有限数量的资源时，有的线程就会因为某一时刻没有空闲的资源而陷入等待。而死锁（Deadlock）就是指这一组中的每一个线程都在等待组内其他线程释放资源而造成的无限等待。

代码片段 9.17 死锁示例

```
1 struct lock lock_A, lock_B;
2
3 void thread_A(void)
4 {
5     lock(&lock_A);
6     // T0 时刻
7     lock(&lock_B);
```